The Practical AI Dictionary

Clear definitions and examples for the AI, automation and agent terminology you keep encountering.

For developers, freelancers, employees, students, job seekers, managers beginning to learn AI, and general AI learners.

Published by Tycho Systems · tychosystem.com

You do not need to memorise this. Keep it open while you read, work or build, and look terms up as you meet them. Each entry gives you a plain-English definition, why the term matters, one practical example, and—where people commonly mix things up—a "do not confuse with" note.

If you only learn ten terms, learn these

A minimum viable vocabulary for following almost any practical AI conversation:

1. **Large language model (LLM)** — the engine inside most current AI tools.
2. **Prompt** — the input you give it; quality in, quality out.
3. **Context window** — how much the model can "see" at once.
4. **Hallucination** — confident, fluent, wrong; the risk to design around.
5. **Workflow** — the unit of real automation: trigger → steps → outcome.
6. **API** — how software talks to software; the doorway into every integration.
7. **RAG** — giving the model your documents to read before it answers.
8. **Structured output** — making the model answer in machine-readable fields.
9. **Human in the loop** — a person approves before anything irreversible happens.
10. **Evaluation** — measuring whether the system is actually good, not just impressive.

Each has a full entry in the pages that follow.

Part 1 · Models and prompting

Artificial intelligence (AI) — Software that performs tasks we associate with human ability: understanding language, recognising images, making predictions. *Why it matters:* it is the umbrella term for everything else in this dictionary. *Example:* a tool that reads an email and suggests a reply is applied AI. *Do not confuse with:* any single product—AI is a category, not a brand.

Machine learning (ML) — Building software that learns patterns from data instead of following hand-written rules. *Why it matters:* it is how modern AI is made; the behaviour comes from training data, not from a programmer specifying every case. *Example:* a spam filter that improves as it sees more examples of spam.

Generative AI — AI that produces new content—text, images, audio, code—rather than only classifying or predicting. *Why it matters:* it is the wave behind ChatGPT, Claude and image generators, and the reason AI moved from back-office prediction into everyday work. *Example:* asking a model to draft a job description.

Large language model (LLM) — A model trained on enormous amounts of text to predict likely next words, which turns out to make it capable of writing, summarising, translating and reasoning in language. *Why it matters:* LLMs are the engine inside most current AI tools. *Example:* GPT, Claude and Gemini are LLMs. *Do not confuse with:* a search engine—an LLM generates text; it does not look facts up unless connected to a data source.

Token — The unit models read and write: a word fragment of roughly four characters in English. *Why it matters:* usage, cost and limits are all counted in tokens. *Example:* "automation" might be two tokens; a page of text is roughly 400–500.

Context window — The maximum amount of text (in tokens) a model can consider at once —your instructions, the conversation so far, and any documents you attach. *Why it matters:* anything outside the window is invisible to the model, which explains why long chats "forget" early details. *Example:* pasting a 300-page contract into a small-window model silently truncates it.

Prompt — The input you give a model: the instruction, the question, the examples, the context. *Why it matters:* output quality tracks prompt quality more than most beginners expect. *Example:* "Summarise this complaint in three bullet points for a support manager" beats "summarise this".

System prompt — Standing instructions that frame every exchange with a model: role, tone, rules, boundaries. Users of a product usually never see it. *Why it matters:* it is how builders make a general model behave like a specific assistant. *Example:* "You are a support assistant for Acme. Never discuss pricing; hand those questions to a human." *Do not confuse with:* the user's prompt—the system prompt sits underneath all of them.

Temperature — A setting that controls how predictable or varied the model's output is. Low = consistent and cautious; high = varied and creative. *Why it matters:* extraction and classification want low temperature; brainstorming tolerates high. *Example:* an invoice-parsing workflow runs at temperature 0.

Hallucination — A model stating something false with complete confidence—invented facts, citations, or numbers. *Why it matters:* it is the core reliability problem to design around; fluency is not accuracy. *Example:* a model citing a court case that does not exist. *Do not confuse with:* a bug—hallucination is inherent to how generative models work and must be managed with validation and review.

Multimodal model — A model that handles more than text: images, audio, sometimes video. *Why it matters:* it lets AI systems read invoices, screenshots and photos directly. *Example:* photographing a receipt and having the model extract the amounts.

Reasoning model — A model variant that spends extra computation "thinking through" a problem step by step before answering. *Why it matters:* better at maths, planning and multi-step logic—slower and costlier for simple tasks. *Example:* using a reasoning model to check a rota against labour rules, but a standard model to draft emails.

Fine-tuning — Additional training that specialises an existing model on your own examples. *Why it matters:* it is often reached for too early; prompting and retrieval (see RAG) usually get there faster and cheaper. *Example:* fine-tuning a model on thousands of past support

replies to match a house style. *Do not confuse with:* RAG—fine-tuning changes the model; RAG changes what it reads.

Inference — Running a trained model to get an output; what you pay for per token when you call an API. *Why it matters:* inference cost and speed drive the economics of any AI feature. *Example:* every chat message triggers inference.

Part 2 · Automation and agents

Automation — Making software perform a task that a person would otherwise do, with or without AI involved. *Why it matters:* most business value comes from ordinary automation with a little AI in the right step—not from AI everywhere. *Example:* new form submission → create CRM record → notify sales channel.

Workflow — A defined sequence of steps that turns a trigger into an outcome. *Why it matters:* it is the unit you should design, measure and improve—"automate the quote follow-up workflow", not "add AI". *Example:* enquiry received → classified → routed → drafted reply → human approves → sent.

Trigger — The event that starts a workflow: a form submission, an email arriving, a schedule, a record change. *Why it matters:* choosing the right trigger determines when and how reliably a workflow runs. *Example:* "every weekday at 07:00" is a schedule trigger; "invoice email received" is an event trigger.

Action — A single step a workflow performs: send a message, create a record, call an API. *Why it matters:* workflows are just triggers plus actions; naming them precisely makes systems easy to reason about. *Example:* "add row to spreadsheet" is one action.

Webhook — A way for one system to notify another the moment something happens, by sending a message to a URL. *Why it matters:* webhooks make integrations instant instead of polling on a timer. *Example:* a payment provider calls your webhook the second a customer pays. *Do not confuse with:* an API call you make—an incoming webhook is a call made to you.

API (application programming interface) — The official, structured way one piece of software talks to another. *Why it matters:* whether a tool has a good API decides whether it can join your automations at all. *Example:* your booking system's API lets a workflow read today's appointments.

Tool use — Giving a model the ability to invoke external functions—search, database lookups, sending messages—instead of only producing text. *Why it matters:* it is the bridge

from "model that talks" to "system that acts". *Example:* an assistant that actually checks the calendar before proposing meeting times.

Function calling — The concrete mechanism for tool use: the model outputs a structured request ("call `create_invoice` with these fields") that your code executes. *Why it matters:* it keeps the model proposing actions while your code stays in control of executing them. *Do not confuse with:* the model executing code itself—your application always makes the call.

AI agent — An AI system that can decide the steps needed to reach a goal, use tools, observe results, and adjust—rather than following a fixed script. *Why it matters:* powerful for open-ended tasks, but harder to make predictable; many "agent" use cases are better served by a plain workflow. *Example:* "find and reconcile mismatched invoices in this folder" handled end to end. *Do not confuse with:* a chatbot—a chatbot converses; an agent takes actions toward a goal.

Agentic workflow — A workflow where the route between defined checkpoints is chosen by a model rather than fixed in advance. *Why it matters:* it is the practical middle ground—model flexibility inside guardrails you control. *Example:* a support flow where the model chooses which knowledge-base articles to consult before drafting, but a human always approves the reply.

Orchestration — Coordinating multiple steps, tools, models and approvals into one dependable process—including order, retries and failure handling. *Why it matters:* production systems live or die on orchestration, not on model choice. *Example:* an orchestrator that runs extraction, validation, database write and notification in sequence, retrying failed steps.

Human in the loop — Designing a workflow so a person reviews or approves the AI's work at defined points. *Why it matters:* it is the standard answer to hallucination and edge cases in business processes. *Example:* AI drafts every quote email; a person clicks send. *Do not confuse with:* a fallback for failures only—human review can be a permanent design choice for consequential steps.

Multi-agent system — Several specialised agents cooperating on a task, often with different roles. *Why it matters:* mostly research and advanced tooling today; useful to

recognise, rarely the right starting point for a small business. *Example:* one agent researches, another writes, a third critiques.

MCP (Model Context Protocol) — An open standard for connecting AI applications to tools and data sources, so any compatible model can use any compatible integration. *Why it matters:* it reduces custom glue code between models and business systems. *Example:* one MCP server for your database serves every MCP-capable AI client you use.

Part 3 · Data and retrieval

Embedding — A list of numbers that captures the meaning of a piece of text (or image), so similar meanings land near each other. *Why it matters:* embeddings are how software compares meaning rather than exact words. *Example:* "my invoice is wrong" and "billing mistake" produce nearby embeddings.

Vector — The general mathematical name for such a list of numbers. *Why it matters:* you will meet the word constantly in AI data tooling; in practice it means "an embedding". *Do not confuse with:* anything mysterious—it is a coordinate list.

Vector database — A database built to store embeddings and find the nearest ones quickly. *Why it matters:* it is the storage layer behind semantic search and RAG. *Example:* storing embeddings of every help article to find the most relevant three for any question.

Semantic search — Search by meaning instead of keyword matching. *Why it matters:* users do not know your exact phrasing; semantic search finds "refund policy" from "can I get my money back". *Example:* an internal search box that finds the right process document however the question is worded.

RAG (retrieval-augmented generation) — Fetching the most relevant pieces of your own data and giving them to the model as context before it answers. *Why it matters:* it grounds a general model in your specific, current information without retraining. *Example:* a support assistant that quotes your actual returns policy because that document was retrieved into its prompt. *Do not confuse with:* fine-tuning—RAG supplies reading material at question time.

Chunking — Splitting documents into pieces before embedding, so retrieval returns focused passages instead of entire files. *Why it matters:* poor chunking is one of the most common reasons a RAG system answers badly. *Example:* splitting a policy manual by section rather than every 1,000 characters mid-sentence.

Knowledge base — The organised collection of documents and facts a system (or team) draws answers from. *Why it matters:* AI answers are only as good as the knowledge base behind them; curation is the real work. *Example:* a maintained set of product specs, policies and how-to guides feeding an assistant.

Structured data — Data with a fixed shape: rows, columns, fields—easy for software to process reliably. *Example:* a spreadsheet of orders with date, customer and amount columns.

Unstructured data — Free-form content: emails, PDFs, call notes, images. *Why it matters:* most business information is unstructured, and turning it into structured data is where AI earns its keep. *Example:* pulling amount and due date out of emailed invoices.

Metadata — Data about data: who created a document, when, what type it is, which customer it concerns. *Why it matters:* good metadata is what makes retrieval, filtering and permissions work at scale. *Example:* tagging each contract with client and renewal date so search can filter by them.

Database — Software for storing data so it can be queried, updated and shared reliably by many users and systems at once. *Why it matters:* spreadsheets quietly become databases in growing businesses—usually badly. *Example:* moving the customer list from a shared spreadsheet into a real database with access control.

SQL — The standard language for asking questions of relational databases. *Why it matters:* it remains the lingua franca of business data, and LLMs are good at writing it—with review. *Example:* `SELECT` all customers with no order in 90 days.

Part 4 · Development and integration

SDK (software development kit) — A ready-made code library for using a service from your programming language, wrapping its API. *Why it matters:* SDKs handle authentication, retries and formatting so you write less error-prone glue code. *Example:* using a provider's Python SDK instead of hand-writing HTTP requests.

REST API — The most common web API style: predictable URLs, standard verbs (GET, POST), usually JSON in and out. *Why it matters:* when a tool says "we have an API", this is usually what it means. *Example:* `GET /customers/42` returns customer 42.

Authentication — Proving who is calling a system—with API keys, tokens or logins. *Why it matters:* every integration starts here, and most "it doesn't work" moments are authentication problems. *Example:* sending an API key in a request header.

OAuth — The standard way to let one app act on your behalf in another without sharing your password, via a consent screen and scoped tokens. *Why it matters:* it is how "Connect your Google account" works, and how access can be limited and revoked. *Do not confuse with:* an API key—OAuth grants scoped, revocable, per-user access.

JSON — The simple text format APIs use for data: names and values in nested braces. *Why it matters:* you will read JSON constantly when building or debugging integrations. *Example:* `{"name": "Ada", "plan": "pro"}`.

Schema — The declared shape data must have: which fields exist, their types, which are required. *Why it matters:* agreeing on schemas is what makes systems interoperate without surprises. *Example:* an order schema requiring `id`, `date` and a positive `amount`.

Structured output — Making a model reply in a machine-readable format (usually JSON matching a schema) instead of prose. *Why it matters:* it is the key technique for putting LLMs inside automations—downstream code needs fields, not paragraphs. *Example:* the model must return `{"category": …, "urgency": …}` for every support email.

Environment variable — A named value (like an API key) supplied to a program by its environment rather than written in the code. *Why it matters:* it is the baseline practice

separating configuration and secrets from source code. *Example:* the server reads `EMAIL_API_KEY` at startup; the key never appears in the repository.

Rate limit — A cap on how many requests a service accepts from you per second, minute or day. *Why it matters:* production systems must expect and handle rate limits, or they fail exactly when busiest. *Example:* an API returning `429 Too Many Requests` during a bulk import.

Idempotency — Designing an operation so doing it twice has the same effect as doing it once. *Why it matters:* networks retry; without idempotency, retries mean duplicate emails, orders and charges. *Example:* using an order ID so a retried "create order" call cannot create a second order.

Queue — A buffer where tasks wait to be processed in order, at a sustainable pace. *Why it matters:* queues absorb spikes and let slow steps (like AI calls) run without blocking everything else. *Example:* incoming documents queue for extraction; a worker processes them one by one.

Retry — Automatically attempting a failed operation again, usually with increasing delays. *Why it matters:* most integration failures are temporary; sensible retries turn flaky into reliable. *Example:* a webhook delivery retried after 1, 4 and 10 minutes. *Do not confuse with:* retrying forever—good systems cap retries and then alert a human.

Part 5 · Reliability and evaluation

Evaluation (evals) — Systematically testing an AI system's outputs against expected results, rather than judging by a few impressive demos. *Why it matters:* it is the difference between "seems good" and "is good"; production AI without evals is guesswork. *Example:* running 200 past support emails through a classifier and scoring its accuracy.

Test set — A collection of inputs with known correct answers, held back for measuring performance. *Why it matters:* it gives you a stable ruler across prompt and model changes. *Example:* fifty real invoices with their hand-verified field values.

Golden dataset — A curated, trusted set of examples representing exactly what "correct" looks like—the standard everything is measured against. *Why it matters:* building it forces the team to agree on what correct even means. *Example:* the twenty trickiest historical enquiries, each with the ideal routing decision attached.

Confidence score — A number a system attaches to an output indicating how sure it is. *Why it matters:* it lets workflows auto-accept the sure cases and route the unsure ones to people. *Example:* extractions below 0.9 confidence go to human review. *Do not confuse with:* correctness—models can be confidently wrong; calibrate scores against real outcomes.

Guardrail — A hard rule constraining what an AI system may do or say, enforced outside the model. *Why it matters:* prompts are suggestions; guardrails are enforcement. *Example:* code that blocks any outgoing reply containing pricing above a threshold, whatever the model wrote.

Validation — Checking data or output against rules before acting on it. *Why it matters:* cheap validation catches most AI mistakes before they become business mistakes. *Example:* rejecting an extracted invoice date in the future.

Observability — Being able to see what a system is doing and why: logs, metrics, traces. *Why it matters:* you cannot fix—or trust—what you cannot see; observability is what separates production systems from demos. *Example:* a dashboard showing each workflow run, its steps, timing and outcome.

Logging — Recording what happened—inputs, decisions, outputs, errors—as the system runs. *Why it matters:* logs are how you diagnose failures and audit decisions after the fact. *Example:* every AI classification stored with its input hash, output and confidence. *Do not confuse with:* logging everything—logs holding personal data need the same care as databases.

Latency — How long a request takes to answer. *Why it matters:* latency decides where AI can sit in an experience—chat tolerates seconds; an inline suggestion does not. *Example:* choosing a smaller model for autocomplete because the big one takes four seconds.

Model drift — Behaviour changing over time—because the provider updated the model or because the world your data describes changed. *Why it matters:* a system that was accurate in March may quietly not be in September; periodic re-evaluation is maintenance, not paranoia. *Example:* re-running the golden dataset after a provider model update.

Prompt versioning — Tracking prompt changes like code changes: numbered, dated, revertible. *Why it matters:* prompts are load-bearing configuration; untracked edits are untraceable behaviour changes. *Example:* rolling back to prompt v14 after v15 tanks the eval score.

Fallback — What the system does when the primary path fails: a backup model, a default answer, or a human. *Why it matters:* reliability comes from designed failure paths, not from hoping nothing fails. *Example:* if extraction fails twice, the document goes to a person with an apologetic note—not into a void.

Part 6 · Privacy and security

Personally identifiable information (PII) — Any data that can identify a person: names, emails, addresses, and combinations that narrow to one individual. *Why it matters:* PII carries legal duties (GDPR and similar) and reputational risk; know where it flows in every workflow—especially into AI tools.

Data minimisation — Collecting and keeping only the data you actually need. *Why it matters:* it is both a legal principle and the cheapest security control—data you never stored cannot leak. *Example:* storing a hash of an email address where the workflow only needs "have we seen this address before".

Encryption — Encoding data so only holders of the key can read it—both in transit (HTTPS) and at rest. *Why it matters:* it is the baseline expectation for any system handling business or customer data.

Least privilege — Every person, key and integration gets the minimum access needed—no more. *Why it matters:* it limits the blast radius when (not if) a credential leaks. *Example:* the reporting workflow's key can read invoices, not modify them.

Secret — Any credential that must not be exposed: API keys, tokens, passwords. *Why it matters:* leaked secrets are one of the most common real-world breaches—never in code, chat logs or prompts. *Example:* rotating an API key immediately after it was pasted into a shared document.

Data retention — Rules for how long data is kept and when it is deleted. *Why it matters:* regulators ask; customers ask; and old data is pure liability. *Example:* deleting lead records eighteen months after the last contact.

Consent — A person's informed, specific agreement to a use of their data—freely given and revocable. *Why it matters:* consent for one purpose (deliver a guide) does not cover another (send marketing); good systems record what was consented to, when, and under which wording. *Example:* a form with a required delivery notice and a separate, unticked marketing checkbox.

Local model — A model running on hardware you control, with data never leaving it. *Why it matters:* the choice for sensitive data or offline use—traded against capability and maintenance burden. *Example:* transcribing internal meetings with a model on your own server.

Cloud model — A model accessed as a service over the internet (the usual case). *Why it matters:* strongest capability with no hardware to run, but data crosses to the provider—check the terms, especially whether your data is used for training. *Do not confuse with:* "public"—business API terms typically differ from consumer chat apps.

A build-first learning path

The fastest way to understand these terms is to meet them in a small real project, in this order:

1. **Learn prompting.** Get reliable results from a chat model on one recurring task you actually do.
2. **Build a simple workflow.** One trigger, a few actions, no AI yet—a form that files itself, a schedule that sends a reminder.
3. **Add an API.** Connect the workflow to one external service and handle its authentication.
4. **Add a structured AI step.** Insert one model call that returns JSON—a classification or extraction—into the workflow.
5. **Add human approval.** Route the AI's output to a person before anything irreversible happens.
6. **Add logging and failure handling.** Record every run; decide what happens on error; add one retry and one fallback.
7. **Build a small real project.** Combine all of it into something a real person (maybe just you) uses weekly. This is the step where the vocabulary becomes experience.

Keep going

Explore practical AI and automation resources from Tycho Systems.

tychosystem.com/resources